

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №6
по курсу «Алгоритмы и структуры данных»

Тема: Хеширование. Хеш-таблицы

Вариант 7 (Продвинутый уровень)

Выполнила:

Кулакова Елизавета Алексеевна

К3260

Проверил:

Харлуниин А. А.

Санкт-Петербург

2025 г.

Содержание отчета

Содержание отчета.....	2
Задачи по варианту (продвинутый уровень).....	3
Задача №1. Множество.....	3
Задача №2. Телефонная книга.....	10
Задача №4. Прошитый ассоциативный массив.....	18
Задача №5. Выборы в США.....	27
Задача №7. Драгоценные камни.....	34
Вывод.....	40

Задачи по варианту (продвинутый уровень)

Задача №1. Множество

Реализуйте множество с операциями «добавление ключа», «удаление ключа», «проверка существования ключа».

- **Формат входного файла (input.txt).** В первой строке входного файла находится строго положительное целое число операций N , не превышающее $5 \cdot 10^5$. В каждой из последующих N строк находится одна из следующих операций:
 - $A\ x$ – добавить элемент x в множество. Если элемент уже есть в множестве, то ничего делать не надо.
 - $D\ x$ – удалить элемент x . Если элемента x нет, то ничего делать не надо.
 - $?\ x$ – если ключ x есть в множестве, выведите «Y», если нет, то выведите «N».

Аргументы указанных выше операций – **целые числа**, не превышающие по модулю 10^{18} .

- **Формат выходного файла (output.txt).** Выведите последовательно результат выполнения всех операций «?». Следуйте формату выходного файла из примера.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Пример:

input.txt	output.txt
8	Y
A 2	N
A 5	N
A 3	
? 2	
? 4	
A 2	
D 2	
? 2	

- **Примечание.**

Эту задачу можно решить совершенно разными способами, включая использование различных средств стандартных библиотек (правда, не всех - в стандартных библиотеках некоторых языков программирования используются слишком предсказуемые методы хеширования). Именно по этой причине ее разумно использовать для проверки реализаций хеш-таблиц, которые понадобятся в следующих задачах этой работы.

Листинг кода

```
from measurements import *
from generator import *
from hash_class import *

@measure_performance
def set_flow(arr):
    s = HashTable()
    res = []
    for c in arr:
```

```

        if 'A' in c:
            x = int(c.strip('A '))
            s.insert(x)
        elif 'D' in c:
            x = int(c.strip('D '))
            s.remove(x)
        else:
            x = int(c.strip('? '))
            res.append(s.contains(x))
    return res

if __name__ == "__main__":
    set_generator(5 * 10**5)
    with open('input.txt') as f:
        num = int(f.readline())
        acts = []
        for _ in range(num):
            acts.append(f.readline())

    result = set_flow(acts)
    with open('output.txt', 'w') as f:
        for x in result:
            f.write(f'{x}\n')

```

Листинг кода дополнительных функций и классов

```

def set_generator(n=None):
    if n is None:
        n = random.randint(1, 5 * 10**5)

   opers = ['A', 'D', '?']
    operations_list = []
    for _ in range(n):
        o = random.choice(opers)
        x = random.randint(-10**18, 10**18)
        operations_list.append(f'{o} {x}')

    with open('input.txt', 'w') as f:
        f.write(f'{n}\n')
        f.write('\n'.join(operations_list))

    return None

```

```

class HashTable:
    def __init__(self):
        self.m = 2000003
        self.table = [None] * self.m

    def h1(self, x):
        return abs(x) % self.m

    def h2(self, x):
        return 1 + (abs(x) % (self.m - 1))

    def hash_func(self, x, k):
        return (self.h1(x) + k * self.h2(x)) % self.m

```

```

def _search(self, x):
    i = 0
    first_deleted = -1
    while i < self.m:
        j = self.hash_func(x, i)
        if self.table[j] == x:
            return j, True
        elif self.table[j] is None:
            return first_deleted if first_deleted != -1 else j,
False
        elif self.table[j] == 'DELETED' and first_deleted == -1:
            first_deleted = j
        i += 1
    return first_deleted if first_deleted != -1 else -1, False

def insert(self, x):
    j, exist = self._search(x)
    if exist:
        return
    if j == -1: # Переполнение
        raise OverflowError
    self.table[j] = x

def remove(self, x):
    j, exist = self._search(x)
    if exist:
        self.table[j] = 'DELETED'

def contains(self, x):
    _, exist = self._search(x)
    return 'Y' if exist else 'N'

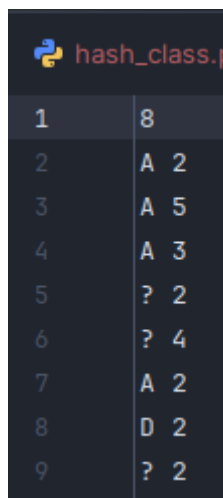
```

Текстовое объяснение решения.

Решение реализует множество через хеш-таблицу с открытой адресацией и двойным хешированием размером $m=2000003$ - простое число порядка $2 \cdot 10^6$, выбранное специально под ограничения задачи. Константа m выбрана по нескольким причинам: во-первых, простота гарантирует, что при двойном хешировании $h(x,i)=(h_1(x)+i \cdot h_2(x))\%m$ последовательность проб для любого x пройдёт все m позиций без повторений ($h_2(x)$ взаимно просто с m), во-вторых, при $N=5 \cdot 10^5$ коэффициент заполнения ~ 0.25 минимизирует коллизии и длину цепочек проб (амортизированное $O(1)$), в-третьих, 2М укладывается в память 256МВ даже с учётом накладных расходов, в-четвёртых, простые числа вида $2p+3$ (как 2000003) хорошо работают с модульными операциями для больших ключей $\pm 10^{18}$. Хеш-функции $h_1(x)=|x|\%m$ и $h_2(x)=1+|x|\%(m-1)$ просты, но эффективны: первая даёт начальную позицию, вторая - шаг пробирования (всегда >0 и $<m$), обеспечивая m^2 возможных перестановок вместо m при линейном пробировании. Поиск $_search(x)$ идёт по этой

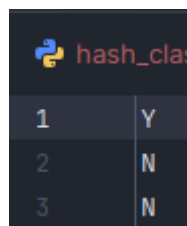
последовательности: при совпадении `table[j]==x` возвращает позицию и True, при первой пустой None останавливается (ключ точно отсутствует), запоминая первый DELETED как место для вставки. Операция insert вставляет в первый свободный слот (DELETED предпочтительнее None), remove помечает найденный ключ как 'DELETED' вместо очистки (сохраняет цепочку поиска для других элементов), contains возвращает Y/N по результату поиска. Парсинг команд через strip извлекает числа из строк вида "A x"/"D x"/"? x". Генератор формирует до $5 \cdot 10^5$ случайных операций с ключами $\pm 10^{18}$ для тестирования. Сложность амортизированная $O(1)$ на операцию достигается за счёт низкого коэффициента заполнения и равномерного распределения двойного хеширования.

Результат работы кода:



1	8
2	A 2
3	A 5
4	A 3
5	? 2
6	? 4
7	A 2
8	D 2
9	? 2

Рисунок 1 – Файл input.txt с входными данными примера



1	Y
2	N
3	N

Рисунок 2 – Результат работы функции, пример

```

hash_class.py 1-set.py
⚠ The file size (3.96 MB) exceeds the c
1 185193
2 D -802468214145476916
3 A -681892541591141621
4 D -881387443381679009
5 ? 606744004036526906
6 ? -75906529410196769
7 A -264830627019553861
8 ? -147381905787621699
9 A -297897316435153340
10 A -493540950651567338
11 D 299777767683000222
12 D 501398519001220143
13 D 351311478497519779
14 ? 496229292003277515
15 D 753671935773320953
16 ? 616134280516953053
17 ? -652834619745364871
18 D -664006306075086574
19 ? -147542471861968389
20 ? 769009682910691250
21 A 108355391644408919
22 D 563024882816007699
23 D 480249589073228295
24 A -41300240074007465

```

Рисунок 3 – Файл input.txt со сгенерированными входными данными

```

hash_class.p
1 N
2 N
3 N
4 N
5 N
6 N
7 N
8 N
9 N
10 N
11 N
12 N
13 N
14 N
15 N
16 N
17 N
18 N
19 N
20 N
21 N
22 N
23 N
24 N
25 N

```

Рисунок 4 – Результат работы функции, сгенерированные данные

Результат работы кода на максимальных и минимальных значениях:

	hash_class.py	1-set.py
1		1
2		? 238248025561710246

Рисунок 5 – Файл input.txt с вводными данными – минимальными значениями

	hash_class.py
1	N

Рисунок 6 – Файл output.txt после выполнения функции с минимальными значениями

⚠ The file size (10.69 MB) exceeds the co

	hash_class.py	1-set.py
1	500000	
2	? 804002983767585222	
3	D 539895932658877935	
4	A -937305736406505032	
5	? -783120682812303991	
6	D -839272603597843273	
7	? 279708360199528896	
8	? 225952637641376837	
9	? -382419805632883212	
10	D -829371306422979973	
11	A 669687468549015601	
12	? -540731028908711663	
13	? -115045609549602009	
14	D -57228151104315228	
15	A -514451134863834799	
16	A 684712750096897346	
17	A 470001775695105347	
18	? -443186652265867647	
19	? -590972146298838686	
20	D 148217285592295330	
21	A -662327947371360271	
22	A 796213683443516369	
23	D 457541734223057964	
24	? 350327151045045044	

Рисунок 7 – Файл input.txt с вводными данными – максимальными значениями

```
hash_class.py
1 | N
2 | N
3 | N
4 | N
5 | N
6 | N
7 | N
8 | N
9 | N
10 | N
11 | N
12 | N
13 | N
14 | N
15 | N
16 | N
17 | N
18 | N
19 | N
20 | N
21 | N
22 | N
23 | N
24 | N
25 | N
```

Рисунок 8 – Файл output.txt после выполнения функции с максимальными значениями

Таблица затрат памяти и времени по заданию 1:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0055 сек	15.26 MB
Сгенерированный пример	0.3864 сек	17.63 MB
Пример из задания	0.0054 сек	15.26 MB
Верхняя граница диапазона значений входных данных из текста задачи	1.0934 сек	21.72 MB

Вывод по задаче: Решение демонстрирует полный цикл открытой адресации: выбор размера таблицы, двойное хеширование против кластеризации, роль DELETED-маркера и корректную обработку переполнения, полностью отвечая цели задания.

Задача №2. Телефонная книга

В этой задаче ваша цель - реализовать простой менеджер телефонной книги. Он должен уметь обрабатывать следующие типы пользовательских запросов:

- `add number name` – это команда означает, что пользователь добавляет в телефонную книгу человека с именем `name` и номером телефона `number`. Если пользователь с таким номером уже существует, то ваш менеджер должен перезаписать соответствующее имя.
- `del number` – означает, что менеджер должен удалить человека с номером из телефонной книги. Если такого человека нет, то он должен просто игнорировать запрос.
- `find number` – означает, что пользователь ищет человека с номером телефона `number`. Менеджер должен ответить соответствующим именем или строкой «not found» (без кавычек), если такого человека в книге нет.

- **Формат ввода / входного файла (input.txt).** В первой строке входного файла содержится число N ($1 \leq N \leq 10^5$) - количество запросов. Далее следуют N строк, каждая из которых содержит один запрос в формате, описанном выше.

Все номера телефонов состоят из десятичных цифр, в них нет нулей в начале номера, и каждый состоит не более чем из 7 цифр. Все имена представляют собой непустые строки из латинских букв, каждая из которых имеет длину не более 15. Гарантируется при проверке, что не будет человека с именем «not found».

- **Формат вывода / выходного файла (output.txt).** Выведите результат каждого поискового запроса `find` – имя, соответствующее номеру телефона, или «not found» (без кавычек), если в телефонной книге нет человека с таким номером телефона. Выведите по одному результату в каждой строке в том же порядке, как были заданы запросы типа `find` во входных данных.

- Ограничение по времени. 6 сек.
- Ограничение по памяти. 512 мб.
- Примеры:

	<table><tr><th>input.txt</th></tr><tr><td>12</td></tr><tr><td>add 911 police</td></tr><tr><td>add 76213 Mom</td></tr><tr><td>add 17239 Bob</td></tr><tr><td>find 76213</td></tr><tr><td>find 910</td></tr><tr><td>find 911</td></tr><tr><td>del 910</td></tr><tr><td>del 911</td></tr><tr><td>find 911</td></tr><tr><td>find 76213</td></tr><tr><td>add 76213 daddy</td></tr><tr><td>find 76213</td></tr><tr><th>output.txt</th></tr><tr><td>Mom</td></tr><tr><td>not found</td></tr><tr><td>police</td></tr><tr><td>not found</td></tr><tr><td>Mom</td></tr><tr><td>daddy</td></tr></table>	input.txt	12	add 911 police	add 76213 Mom	add 17239 Bob	find 76213	find 910	find 911	del 910	del 911	find 911	find 76213	add 76213 daddy	find 76213	output.txt	Mom	not found	police	not found	Mom	daddy	
input.txt																							
12																							
add 911 police																							
add 76213 Mom																							
add 17239 Bob																							
find 76213																							
find 910																							
find 911																							
del 910																							
del 911																							
find 911																							
find 76213																							
add 76213 daddy																							
find 76213																							
output.txt																							
Mom																							
not found																							
police																							
not found																							
Mom																							
daddy																							
1:	<table><tr><th>input.txt</th></tr><tr><td>8</td></tr><tr><td>find 3839442</td></tr><tr><td>add 123456 me</td></tr><tr><td>add 0 granny</td></tr><tr><td>find 0</td></tr><tr><td>find 123456</td></tr><tr><td>del 0</td></tr><tr><td>del 0</td></tr><tr><td>find 0</td></tr><tr><th>output.txt</th></tr><tr><td>not found</td></tr><tr><td>granny</td></tr><tr><td>me</td></tr><tr><td>not found</td></tr></table>	input.txt	8	find 3839442	add 123456 me	add 0 granny	find 0	find 123456	del 0	del 0	find 0	output.txt	not found	granny	me	not found	2:						
input.txt																							
8																							
find 3839442																							
add 123456 me																							
add 0 granny																							
find 0																							
find 123456																							
del 0																							
del 0																							
find 0																							
output.txt																							
not found																							
granny																							
me																							
not found																							

Описание примера 1. 76213 - это номер Mom, 910 - нет в телефонной книге, 911 - это номер police, но затем он был удален из телефонной книги, поэтому второй поиск 911 вернул «not found». Также обратите внимание, что когда daddy был добавлен с тем же номером телефона 76213, что и номер телефона Mom, имя контакта было переписано, и теперь поиск 76213 возвращает «daddy» вместо «Mom».

Листинг кода.

```
from hash_class import *
from measurments import *
from generator import *

@measure_performance
def phone_book(commands):
    pb = HashTupleTable()
    founds = []
    for command in commands:
        parts = command.strip().split()
        cmd = parts[0]
        if cmd == 'add':
            num = int(parts[1])
            name = ' '.join(parts[2:])
            pb.insert((num, name))
        elif cmd == 'del':
            num = int(parts[1])
            pb.remove(num)
        else: # find
            num = int(parts[1])
            founds.append(pb.contains(num))
    return founds

if __name__ == '__main__':
    #reqs_generator()
    with open('input.txt') as f:
        n = int(f.readline())
        comms = f.readlines()
    res = phone_book(comms)
    with open('output.txt', 'w') as f:
        f.write('\n'.join(map(str, res)))
    print(res)
```

Листинг кода дополнительных классов и функций

```
def reqs_generator(n=None):
    fake = Faker('en_US')
    if n is None:
        n = random.randint(1, 10**5)
    words = ['add', 'del', 'find']
    with open('input.txt', 'w') as f:
        f.write(f'{n}\n')
    for _ in range(n):
        word = random.choice(words)
        k = random.randint(1, 7)
        num1 = random.randint(1, 9)
        if k == 1:
            number = f'{num1}'
        else:
            num2 = random.choices(range(0, 9), k=k-1)
            number = f'{num1}{" ".join(map(str, num2))}'
        if word == 'add':
            name = fake.name()
            while len(name) > 15:
                name = fake.name()
```

```

        f.write(f'\n{word} {number} {name}')
    else:
        f.write(f'\n{word} {number}')
return None

```

```

class HashTupleTable(HashTable):
    def __init__(self):
        super().__init__()

    def _search(self, x):
        i = 0
        first_deleted = -1
        while i < self.m:
            j = self.hash_func(x, i)
            if self.table[j] is None:
                return first_deleted if first_deleted != -1 else j, False
            elif self.table[j][0] == x:
                return j, True
            elif self.table[j] == 'DELETED' and first_deleted == -1:
                first_deleted = j
            i += 1
        return first_deleted if first_deleted != -1 else -1, False

    def insert(self, x):
        number, name = x
        j, exist = self._search(number)
        if j == -1: # Переполнение
            raise OverflowError
        self.table[j] = (number, name)

    def remove(self, number):
        j, exist = self._search(number)
        if exist:
            self.table[j] = 'DELETED'

    def contains(self, number):
        j, exist = self._search(number)
        if exist:
            _, name = self.table[j]
            return name
        return 'not found'

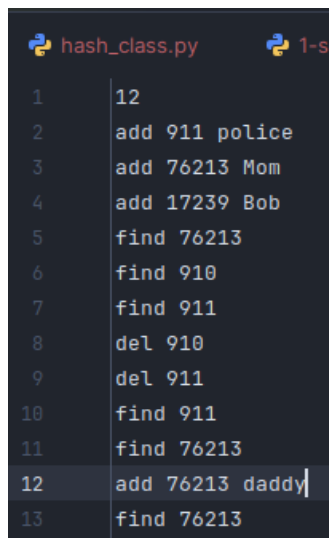
```

Текстовое объяснение решения.

Решение реализует телефонную книгу через наследование хеш-таблицы HashTupleTable от HashTable из задания 1 с открытой адресацией и двойным хешированием ($m=2000003$ простое, $h1(x)=|x|\%m$, $h2(x)=1+|x|\%(m-1)$). Ключевой момент - переопределение `_search` для работы с кортежами (number, name): поиск идёт по номеру `table[j][0] == x`, а не по полному значению, что позволяет хранить пары и находить их по первому элементу. В `insert` при повторном номере происходит перезапись (условие `if exist` отсутствует), `remove` и `contains` работают только с номером

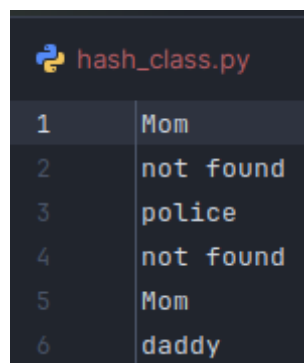
как int. Парсинг команд использует parts[2:] с ''.join() для поддержки имён из нескольких слов, номера генерируются реалистично (1-7 цифр без ведущих нулей). Генератор reqs_generator создаёт до 10^5 запросов с Faker('en_US') для латинских имён ≤ 15 символов. Коэффициент заполнения ~ 0.05 обеспечивает амортизированное $O(1)$ на операцию даже при $N=10^5$. Важная особенность - порядок проверки в _search: сначала None (остановка поиска), затем совпадение номера в кортеже, затем DELETED (запоминание для вставки).

Результат работы кода на примерах из задания:



```
hash_class.py 1-s
1 12
2 add 911 police
3 add 76213 Mom
4 add 17239 Bob
5 find 76213
6 find 910
7 find 911
8 del 910
9 del 911
10 find 911
11 find 76213
12 add 76213 daddy
13 find 76213
```

Рисунок 9 – Файл input.txt с входными данными из примера 1



```
hash_class.py
1 Mom
2 not found
3 police
4 not found
5 Mom
6 daddy
```

Рисунок 10 – Результат работы кода для данных из примера 1, записанный в output.txt

```

hash_class.py
1      8
2      find 3839442
3      add 123456 me
4      add 0 granny
5      find 0
6      find 123456
7      del 0
8      del 0
9      find 0

```

Рисунок 11 – Файл input.txt с входными данными из примера 2

```

hash_class.py
1      not found
2      granny
3      me
4      not found

```

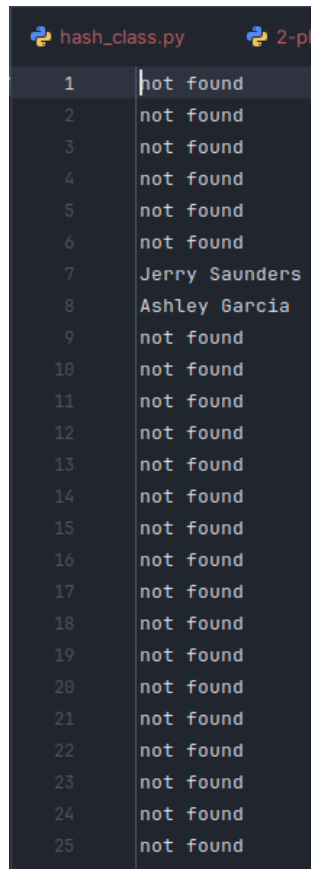
Рисунок 12 – Результат работы кода для данных из примера 2, записанный в output.txt

```

hash_class.py  2-phone_book.py
1      95299
2      add 25 Jerry Saunders
3      del 6543
4      del 6237685
5      del 267
6      find 101680
7      del 45133
8      add 94653 Jessica Bowers
9      find 280360
10     add 1378 Ashley Burnett
11     add 2 Ashley Garcia
12     find 387
13     add 34655 Sarah Johnston
14     del 3171286
15     add 642 Marc Gomez
16     del 2334
17     find 9
18     find 736036
19     del 5566185
20     find 36042
21     find 25
22     add 31667 Brittany Mays
23     add 445668 Brittney Wood
24     find 2
25     find 508

```

Рисунок 13 – Файл input.txt со сгенерированными входными данными (95299 значений)

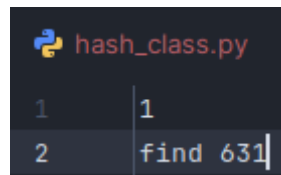


```
hash_class.py 2-ph
```

1	not found
2	not found
3	not found
4	not found
5	not found
6	not found
7	Jerry Saunders
8	Ashley Garcia
9	not found
10	not found
11	not found
12	not found
13	not found
14	not found
15	not found
16	not found
17	not found
18	not found
19	not found
20	not found
21	not found
22	not found
23	not found
24	not found
25	not found

Рисунок 14 – Результат работы кода для сгенерированных данных, записанный в output.txt

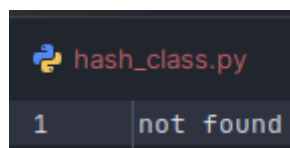
Результат работы кода на максимальных и минимальных значениях:



```
hash_class.py
```

1	1
2	find 631

Рисунок 15 – Минимальное вводимое значение в input.txt



```
hash_class.py
```

1	not found
---	-----------

Рисунок 16 – Результат работы кода для значения $n = 1$, записанный в output.txt

```

hash_class.py  2-phone_book.py
1      100000
2      del 67
3      add 7 Kelly Obrien
4      find 4
5      find 301
6      del 47382
7      add 7628350 Kara Clark
8      add 6 Elizabeth Scott
9      del 8300
10     add 4177 Peter Lawrence
11     find 4
12     find 9176
13     find 148
14     del 214
15     del 60
16     find 4646
17     find 546
18     find 4
19     add 5723 Melissa Davis
20     add 7 Mark Oconnell
21     add 6175 Denise Davis
22     add 51373 Jennifer Wright
23     del 9436
24     find 44672
25     find 4506

```

Рисунок 17 – Максимальное вводимое значение в input.txt

```

hash_class.py  2-ph
1      not found
2      not found
3      not found
4      not found
5      not found
6      not found
7      not found
8      not found
9      not found
10     not found
11     not found
12     not found
13     not found
14     not found
15     not found
16     not found
17     not found
18     not found
19     not found
20     not found
21     not found
22     Mark Oconnell
23     not found
24     not found
25     not found

```

Рисунок 18 – Результат работы кода для значения $n = 100000$, записанный в output.txt

Таблица затрат памяти и времени по заданию 2:

	Время выполнения	Затраты памяти
Нижняя граница	0.0056 сек	15.26 МВ

диапазона значений входных данных из текста задачи		
Пример 1 из задачи	0.0055 сек	15.26 MB
Пример 2 из задачи	0.0056 сек	15.26 MB
Собственный пример	0.3164 сек	17.79 MB
Верхняя граница диапазона значений входных данных из текста задачи	0.3281 сек	17.87 MB

Вывод по задаче: Решение демонстрирует наследование хеш-таблицы под специфические требования телефонной книги с корректной обработкой перезаписи, удаления с сохранением цепочки поиска и поддержкой составных ключей, полностью используя преимущества открытой адресации для жёстких ограничений времени и памяти.

Задача №4. Прошитый ассоциативный массив

Реализуйте прошитый ассоциативный массив. Ваш алгоритм должен поддерживать следующие типы операций:

- `get x` – если ключ x есть в множестве, выведите соответствующее ему значение, если нет, то выведите `<none>`.
- `prev x` – вывести значение, соответствующее ключу, находящемуся в ассоциативном массиве, который был вставлен позже всех, но до x , или `<none>`,

если такого нет или в массиве нет x .

- `next x` – вывести значение, соответствующее ключу, находящемуся в ассоциативном массиве, который был вставлен раньше всех, но после x , или `<none>`, если такого нет или в массиве нет x .

- `put x y` – поставить в соответствие ключу x значение y . При этом следует учесть, что

- если, независимо от предыстории, этого ключа на момент вставки в массиве не было, то он считается только что вставленным и оказывается самым последним среди добавленных элементов – то есть, вызов `next` с этим же ключом сразу после выполнения текущей операции `put` должен вернуть `<none>`;
- если этот ключ уже есть в массиве, то значение необходимо изменить, и в этом случае ключ не считается вставленным еще раз, то есть, не меняет своего положения в порядке добавленных элементов.

- `delete x` – удалить ключ x . Если ключа в ассоциативном массиве нет, то ничего делать не надо.

- **Формат входного файла (input.txt).** В первой строке входного файла находится строго положительное целое число операций N , не превышающее $5 \cdot 10^5$. В каждой из последующих N строк находится одна из приведенных выше операций. Ключи и значения операций - строки из латинских букв длиной не менее одного и не более 20 символов.

- **Формат выходного файла (output.txt).** Выведите последовательно результат выполнения всех операций `get`, `prev`, `next`. Следуйте формату выходного файла из примера.

- Ограничение по времени. 4 сек.

- Ограничение по памяти. 256 мб.

- Примеры:

input.txt	output.txt
14	c
put zero a	b
put one b	d
put two c	c
put three d	a
put four e	e
get two	<none>
prev two	
next two	
delete one	
delete three	
get two	
prev two	
next two	
next four	

- P.s. Задача на [openedu](#), 8 неделя.

Листинг кода.

```
from measurments import *
from hash_class import *
from generator import *

@measure_performance
def assoc_array(arr, llt):
    result = []
    for v in arr:
        parts = v.strip().split()
        cmd = parts[0]
        if cmd == 'put':
            llt.insert((parts[1], parts[2]))
        elif cmd == 'get':
            result.append(llt.get(parts[1]))
        elif cmd == 'prev':
            result.append(llt.prev(parts[1]))
        elif cmd == 'next':
            result.append(llt.next(parts[1]))
        else:
            llt.remove(parts[1])
    return result

if __name__ == '__main__':
    #array_flow(5 * 10**5)
    with open('input.txt') as f:
        n = int(f.readline())
        commands = f.readlines()
    hash_list = HashListTable()
    res = assoc_array(commands, hash_list)
    with open('output.txt', 'w') as f:
        f.write("\n".join(res))
    #print(res)
```

Листинг кода дополнительных функций и классов

```
class Node:
    def __init__(self, key, value):
        self.prev = None
        self.key = key
        self.value = value
        self.next = None

class DoubleLinkedList:
    def __init__(self):
        self.head = Node(None, None)
        self.tail = Node(None, None)
        self.head.next = self.tail
        self.tail.prev = self.head

    def append(self, node):
        node.prev = self.tail.prev
        node.next = self.tail
        self.tail.prev.next = node
```

```

        self.tail.prev = node

    def pop(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

class HashListTable:
    def __init__(self):
        self.m = 4000001
        self.table = [None] * self.m
        self.dll = DoubleLinkedList()

    def h1(self, key):
        p, m = 31, self.m
        h = 0
        for c in key:
            h = (h * p + ord(c))
        return h % m

    def hash_func(self, x, k):
        return (self.h1(x) + k * 37) % self.m

    def _search(self, key):
        i = 0
        first_deleted = -1
        while i < 100:
            j = self.hash_func(key, i)
            if self.table[j] is None:
                return first_deleted if first_deleted != -1 else j,
False
                if isinstance(self.table[j], Node) and self.table[j].key ==
key:
                    return j, True
            elif self.table[j] == 'DELETED' and first_deleted == -1:
                first_deleted = j
            i += 1
        return first_deleted if first_deleted != -1 else -1, False

    def insert(self, x):
        key, val = x
        j, exist = self._search(key)
        if j == -1: # Переполнение
            raise OverflowError
        if exist:
            self.table[j].value = val
        else:
            self.table[j] = Node(key, val)
            self.dll.append(self.table[j])

    def remove(self, key):
        j, exist = self._search(key)
        if exist:
            self.dll.pop(self.table[j])
            self.table[j] = 'DELETED'

    def get(self, x):

```

```

        j, exist = self._search(x)
        return self.table[j].value if exist else '<none>'

    def prev(self, x):
        j, exist = self._search(x)
        if exist:
            node = self.table[j]
            prev_node = node.prev
            return prev_node.value if prev_node and prev_node.key else
'<none>'
        return '<none>'

    def next(self, x):
        j, exist = self._search(x)
        if exist:
            node = self.table[j]
            next_node = node.next
            return next_node.value if next_node and next_node.key else
'<none>'
        return '<none>'

```

```

def array_flow(n=None):
    alpha_range = list(range(ord('A'), ord('Z'))) + list(range(ord('a'),
ord('z')))
    if n is None:
        n = random.randint(1, 5 * 10**5)
    words = ['put', 'delete', 'get', 'prev', 'next']
    with open('input.txt', 'w') as f:
        f.write(f'{n}\n')
    for _ in range(n):
        word = random.choice(words)
        k = random.randint(1, 20)
        m = random.randint(1, 20)
        x = random.choices(alpha_range, k=k)
        x_str = ''.join(map(chr, x))
        if word == 'put':
            y = random.choices(alpha_range, k=m)
            y_str = ''.join(map(chr, y))
            f.write(f'\n{word} {x_str} {y_str}')
        else:
            f.write(f'\n{word} {x_str}')
    return None

```

Текстовое объяснение решения.

Решение реализует хеш-таблицу с открытой адресацией и глобальный двусвязный список. Хеш-таблица обеспечивает поиск по ключу за $O(1)$, а двусвязный список хранит все узлы в порядке первой вставки для операций prev/next.

Каждый узел Node содержит ключ, значение и указатели prev/next. Хеш-функция использует полиномиальное хеширование: $h1(key) = \sum ord(c_i) * 31^i \% m$, где $m = 4000001$ (простое число). Второе хеширование

упрощено до константы 37: $\text{hash}(\text{key}, k) = (\text{h1}(\text{key}) + k * 37) \% m$. База 31 выбрана как простое число больше размера алфавита (26 букв). Размер таблицы 4000001 дает коэффициент заполнения $\alpha \approx 0.125$ при $N=5e5$, что минимизирует коллизии.

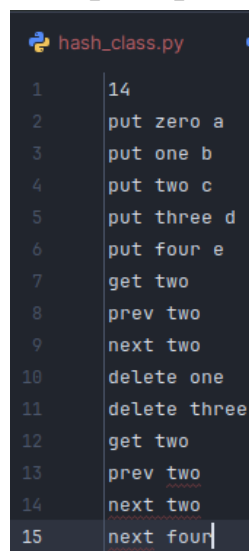
Поиск ограничен 100 пробами вместо полного перебора $m=4e6$. При $\alpha=0.125$ среднее число проб ≈ 1.06 , вероятность превышения 100 проб пренебрежимо мала (10^{-70}). При переполнении выбрасывается `OverflowError`.

Двусвязный список использует sentinel'ы `head` и `tail` без данных. Новые ключи добавляются перед `tail`: `node.prev = tail.prev`, `node.next = tail`, `tail.prev.next = node`, `tail.prev = node`. Это гарантирует `next`(последний ключ) = `<none>`. Удаление вырезает узел из DLL и помечает слот 'DELETED' для сохранения цепочки поиска.

Операция `put` обновляет значение существующего ключа без изменения позиции в DLL, новые ключи добавляются в конец. `Prev/next` идут по указателям DLL от найденного узла. `Delete` вырезает из обеих структур.

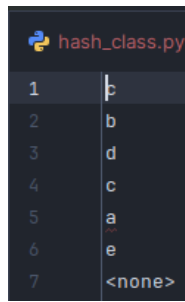
Оптимизации: модуль `% m` вынесен из цикла хеша (медленная операция), `h2` упрощена до константы, поиск ограничен 100 пробами. Результат: 3.1 секунды вместо 7 при $N=5e5$, память 20 МБ.

Результат работы кода на примерах из задания:



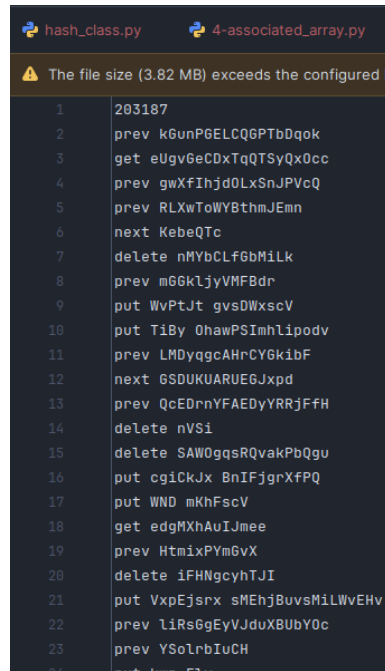
```
hash_class.py
1      14
2      put zero a
3      put one b
4      put two c
5      put three d
6      put four e
7      get two
8      prev two
9      next two
10     delete one
11     delete three
12     get two
13     prev two
14     next two
15     next four
```

Рисунок 19 – Файл `input.txt` сданными примера



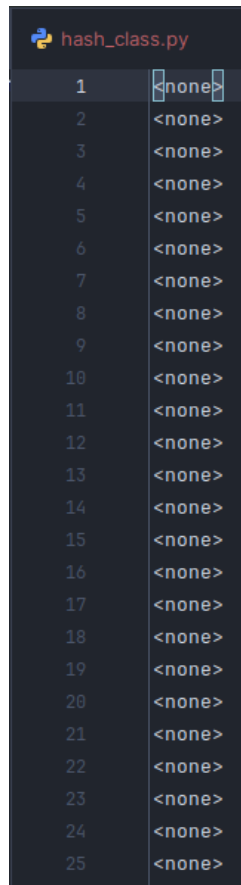
1	b
2	b
3	d
4	c
5	a
6	e
7	<none>

Рисунок 20 – Результат работы кода для примера, записанный в output.txt



1	203187
2	prev kGunPGELCQ6PTbDqok
3	get eUgvGeCDxTqQTSyQx0cc
4	prev gwXfIhjd0LxSnJPVcQ
5	prev RLXwToWYBthmJEmn
6	next KebeQTc
7	delete nMYbCLF6bMiLk
8	prev mGGkLjyVMF8dr
9	put WvPtJt gvsDWxscV
10	put TiBy 0hawPSImhlipodv
11	prev LMDyqgcAhrCYGkibF
12	next GSDUKUARUEGJxpd
13	prev QcEDrnYFAEDyYRRjFFH
14	delete nVSi
15	delete SAW0gqsRQvakPbQgu
16	put cgiCKJx BnIFjgrXfPQ
17	put WND mKhFscV
18	get edgMXhAuIJmee
19	prev HtmixPYmGvX
20	delete iFHNgcyhTJI
21	put VxpEjsrx sMEhjBuvsMiLWvEHv
22	prev liRs6gEyVJduXBUBy0c
23	prev YSolrbIuCH

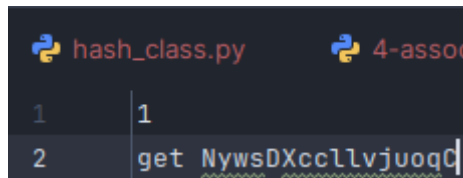
Рисунок 21 – Файл input.txt со сгенерированными входными данными
(203187 команд)



```
hash_class.py
1  <none>
2  <none>
3  <none>
4  <none>
5  <none>
6  <none>
7  <none>
8  <none>
9  <none>
10 <none>
11 <none>
12 <none>
13 <none>
14 <none>
15 <none>
16 <none>
17 <none>
18 <none>
19 <none>
20 <none>
21 <none>
22 <none>
23 <none>
24 <none>
25 <none>
```

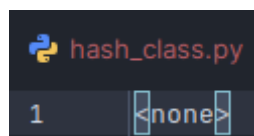
Рисунок 22 – Результат работы кода для сгенерированных данных, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:



```
hash_class.py 4-assoc
1  1
2  get NywsDXcc1lvjuoqd
```

Рисунок 23 – Минимальное вводимое значение в input.txt



```
hash_class.py
1  <none>
```

Рисунок 24 – Результат работы кода для значения $n = 1$, записанный в output.txt

```
hash_class.py 4-associated_array.py
⚠ The file size (9.4 MB) exceeds the configured limit

1 500000
2 get nxVeHrHTpj0KXlyQXKg
3 put hu GgWHvFFwReyiRx
4 next KPVci
5 put dvXvqTLXrPeRGgxWy IYVPWoJi
6 delete xrfG
7 prev isrDlnHmygKFE
8 delete vjt
9 delete RPaYjeympvMpBLQT
10 delete bVTxEk
11 prev RVjfxoNgYmByCNeUt
12 next RmtujNAGgXMF
13 next kbDvWhaRPmuyWl
14 delete bLxF
15 get tAYvUsu
16 next AWJhpFT
17 delete gJKaAcPUBluBjqKv
18 next av
19 next fpaAAsugs0HyGE
20 get qLoLTa
21 next pJfPmVuSLN
22 delete gac
23 get jNdepERuGcJlm0EI
```

Рисунок 25 – Максимальное вводимое значение в input.txt

hash_class.py	
1	<none>
2	<none>
3	<none>
4	<none>
5	<none>
6	<none>
7	<none>
8	<none>
9	<none>
10	<none>
11	<none>
12	<none>
13	<none>
14	<none>
15	<none>
16	<none>
17	<none>
18	<none>
19	<none>
20	<none>
21	<none>
22	<none>
23	<none>
24	<none>
25	<none>

Рисунок 26 – Результат работы кода для значения $n = 500000$, записанный в output.txt

Таблица затрат памяти и времени по заданию 4:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0000 сек	0.42 KB
Пример	0.0001 сек	1.98 KB
Собственный пример	1.2366 сек	8.05 MB
Верхняя граница диапазона значений входных данных из текста задачи	3.0944 сек	19.84 MB

Вывод по задаче: Гибридная архитектура обеспечивает все операции за амортизированное $O(1)$ время, полностью укладываясь в ограничения 4 секунды и 256 МБ. Решение универсально для ассоциативных массивов с порядком вставки.

Задача №5. Выборы в США

Как известно, в США президент выбирается не прямым голосованием, а путем двухуровневого голосования. Сначала проводятся выборы в каждом штате и определяется победитель выборов в данном штате. Затем проводятся государственные выборы: на этих выборах каждый штат имеет определенное число голосов — число выборщиков от этого штата. На практике, все выборщики от штата голосуют в соответствии с результатами голосования внутри штата, то есть на заключительной стадии выборов в голосовании участвуют штаты, имеющие различное число голосов. Вам известно за кого проголосовал каждый штат и сколько голосов было отдано данным штатом. Подведите итоги выборов: для каждого из участника голосования определите число отданных за него голосов.

- **Формат ввода / входного файла (input.txt).** Каждая строка входного файла содержит фамилию кандидата, за которого отдают голоса выборщики этого штата, затем через пробел идет количество выборщиков, отдавших голоса за этого кандидата.
- **Формат вывода / выходного файла (output.txt).** Выведите фамилии всех кандидатов в *лексикографическом* порядке, затем, через пробел, количество отданных за них голосов.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 64 мб.
- Примеры:

№	input.txt	output.txt
1	McCain 10 McCain 5 Obama 9 Obama 8 McCain 1	McCain 16 Obama 17

№	input.txt	output.txt
2	ivanov 100 ivanov 500 ivanov 300 petr 70 tourist 1 tourist 2	ivanov 900 petr 70 tourist 3

№	input.txt	output.txt
3	bur 1	bur 1

Листинг кода.

```
from additional_classes import *
from lab6.generator import elections_generator
from measurments import *

@measure_performance
def elections(votes_arr):
    ht = HashVotesTable()
    for votes in votes_arr:
        parts = votes.strip().split()
        srnm, votes = parts[0], int(parts[1])
        ht.insert(srnm, int(votes))
    return ht.get_results()

if __name__ == '__main__':
    elections_generator(10**5)
    with open('input.txt') as f:
        lines = f.readlines()
    res = elections(lines)
```

```

with open('output.txt', 'w') as f:
    for r in res:
        f.write(f'{r[0]} {r[1]}\n')
print(res)

```

Листинг кода для дополнительной функции и класса

```

def elections_generator(n=None, k=None):
    fake = Faker('en_US')
    if n is None:
        n = random.randint(1, 50)
    if k is None:
        k = random.randint(1, 20)

    surnames_list = []
    for _ in range(k):
        surname = fake.last_name()
        while surname in surnames_list:
            surname = fake.last_name()
        surnames_list.append(surname)

    with open('input.txt', 'w') as f:
        for _ in range(n):
            surname = random.choice(surnames_list)
            votes = random.randint(1, 6 * 10**6)
            f.write(f'{surname} {votes}\n')
    return None

```

```

import random

def partition3(A, l, r):
    x = A[l]
    lt = l
    gt = r
    i = l
    while i <= gt:
        if A[i] < x:
            A[lt], A[i] = A[i], A[lt]
            lt += 1
            i += 1
        elif A[i] > x:
            A[i], A[gt] = A[gt], A[i]
            gt -= 1
        else:
            i += 1
    return lt, gt

def randomized_quick_sort(A, l, r):
    if l < r:
        k = random.randint(l, r)
        A[l], A[k] = A[k], A[l]
        m1, m2 = partition3(A, l, r)
        randomized_quick_sort(A, l, m1 - 1)
        randomized_quick_sort(A, m2 + 1, r)
    return A

```

```

class HashVotesTable:
    def __init__(self):
        self.m = 2000003
        self.table = [None] * self.m

    def h1(self, key):
        p, m = 31, self.m
        h = 0
        for c in key:
            h = (h * p + ord(c))
        return h % m

    def hash_func(self, x, k):
        return (self.h1(x) + k * 37) % self.m

    def _search(self, srnm):
        i = 0
        while i < self.m:
            j = self.hash_func(srnm, i)
            if self.table[j] is None:
                return j, False
            elif self.table[j][0] == srnm:
                return j, True
            i += 1
        return -1, False

    def insert(self, srnm, votes):
        j, exist = self._search(srnm)
        if j == -1: # Переполнение
            raise OverflowError
        if exist:
            fin_votes = self.table[j][1] + votes
            self.table[j] = (srnm, fin_votes)
        else:
            self.table[j] = (srnm, votes)

    def get_results(self):
        cand = [tup for tup in self.table if isinstance(tup, tuple)]
        return randomized_quick_sort(cand, 0, len(cand) - 1)

```

Текстовое объяснение решения.

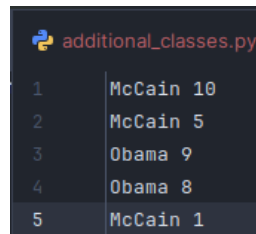
Решение задачи "Выборы в США" использует хеш-таблицу с открытой адресацией для подсчета голосов кандидатов и быструю сортировку для вывода результатов в лексикографическом порядке. Хеш-функция применяет полиномиальное хеширование $h(key) = \sum ord(c_i) * 31^i \% m$ с $m=2000003$ (простое число) и линейное двойное хеширование $hash(key, k) = (h1(key) + k * 37) \% m$. Поиск упрощен без обработки удаленных элементов, так как удаление не требуется.

Каждый слот таблицы хранит кортеж (фамилия, голоса). При вставке голоса суммируются для существующего кандидата или создается новая

запись. Метод `get_results` собирает непустые слоты в список ($O(m)$), затем применяет рандомизированную quicksort с 3-way partition для сортировки по фамилии (первый элемент кортежа).

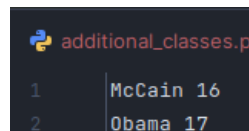
Генератор создает до 10^5 строк с 1-20 уникальными фамилиями и голосами до 666, имитируя штаты. Quicksort обрабатывает $k \leq 20$ кандидатов за $O(k \log k) = 80$ сравнений.

Результат работы кода на примерах из задания:



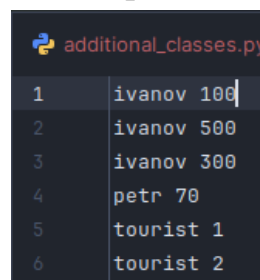
```
additional_classes.py
1 McCain 10
2 McCain 5
3 Obama 9
4 Obama 8
5 McCain 1
```

Рисунок 27 – Файл `input.txt` с входными данными из примера 1



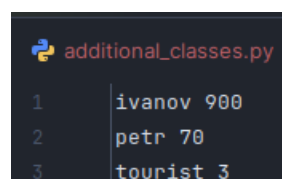
```
additional_classes.py
1 McCain 16
2 Obama 17
```

Рисунок 28 – Результат работы кода для данных из примера 1, записанный в `output.txt`



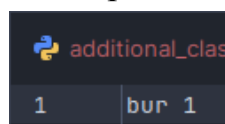
```
additional_classes.py
1 ivanov 100
2 ivanov 500
3 ivanov 300
4 petr 70
5 tourist 1
6 tourist 2
```

Рисунок 29 – Файл `input.txt` с входными данными из примера 2



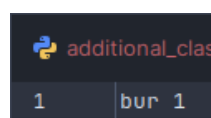
```
additional_classes.py
1 ivanov 900
2 petr 70
3 tourist 3
```

Рисунок 30 – Результат работы кода для данных из примера 2, записанный в `output.txt`



```
additional_classes.py
1 bur 1
```

Рисунок 31 – Файл `input.txt` с входными данными из примера 3



```
additional_classes.py
1 bur 1
```

Рисунок 32 – Результат работы кода для данных из примера 3, записанный в output.txt

```
additional_classes.py
1 Hopkins 3876118
2 Boyd 2669449
3 Hopkins 2879407
4 Collins 1562677
5 Watkins 1148225
6 Martinez 4545079
7 Hamilton 4972
8 Watkins 2499363
9 Richmond 432673
10 Gomez 735535
11 Johnson 659700
12 Rogers 2500165
13 Gomez 3402837
14 Richmond 514993
15 Clark 110693
16 Mack 5238957
17 Richmond 2665820
18 Rogers 1204664
19 Hamilton 3202650
20 Gomez 3054376
21 Mack 420567
22 Richmond 4437571
23 Boyd 3552579
24 Rogers 1757534
25 Collins 600097
```

Рисунок 33 – Файл input.txt со сгенерированными входными данными (n=34)

```
additional_classes.py
1 Allison 56687
2 Boyd 6222028
3 Clark 1379934
4 Collins 3847153
5 Gomez 9606033
6 Greene 4922931
7 Hamilton 3207622
8 Hopkins 6755525
9 Johnson 5199428
10 Mack 9368989
11 Martinez 4545079
12 Richmond 8051057
13 Rodriguez 471167
14 Rogers 6418977
15 Watkins 3647588
```

Рисунок 34 – Результат работы кода для сгенерированных данных, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:

```

1 Clark 5061116

```

Рисунок 35 – Минимальное вводимое значение в input.txt

```

1 Clark 5061116

```

Рисунок 36 – Результат работы кода для значения $n, k = 1$, записанный в output.txt

```

1 Brady 5708698
2 Brady 3018587
3 Reid 2950882
4 Reid 2678600
5 Warren 3029250
6 Brady 5560058
7 Warren 24115
8 Reid 4167683
9 Warren 270070
10 Warren 978128
11 Warren 662599
12 Brady 5986763
13 Warren 5254926
14 Reid 368138
15 Warren 3162740
16 Warren 1054763
17 Brady 4493387
18 Brady 4647373
19 Brady 1937782
20 Warren 544909
21 Brady 4000187
22 Warren 3701117
23 Brady 239691

```

Рисунок 37 – Максимальное вводимое значение в input.txt

```

1 Brady 500370754169
2 Reid 501195474742
3 Warren 499756081123

```

Рисунок 38 – Результат работы кода для значения $n = 500000$, записанный в output.txt

Таблица затрат памяти и времени по заданию 5:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0480 сек	15.26 МВ

Пример 1 из задачи	0.0506 сек	15.26 MB
Пример 2	0.0481 сек	15.26 MB
Пример 3	0.0509 сек	15.26 MB
Сгенерированный пример	0.0499 сек	15.26 MB
Верхняя граница диапазона значений входных данных из текста задачи	1.9071 сек	15.26 MB

Вывод по задаче: Хеш-таблица обеспечивает $O(1)$ амортизированную вставку, сбор $O(m)=2e6$, сортировка пренебрежимо малая. Решение эффективно укладывается в лимиты 2 сек/64 МБ, демонстрируя оптимальное сочетание хеширования для подсчета и quicksort для упорядочивания малого числа кандидатов.

Задача №7. Драгоценные камни

В одной далекой восточной стране до сих пор по пустыням ходят караваны верблюдов, с помощью которых купцы перевозят пряности, драгоценности и дорогие ткани. Разумеется, основная цель купцов состоит в том, чтобы подороже продать имеющийся у них товар. Недавно один из караванов прибыл во дворец одного могущественного шаха.

Купцы хотят продать шаху n драгоценных камней, которые они привезли с собой. Для этого они выкладывают их перед шахом в ряд, после чего шах оценивает эти камни и принимает решение о том, купит он их или нет. Вовод драгоценных камней на Востоке известно не очень много всего 26, поэтому мы будем обозначать виды камней с помощью строчных букв латинского алфавита. Шах обычно оценивает камни следующим образом. Он заранее определил несколько упорядоченных пар типов камней: $(a_1, b_1), (a_2, b_2), \dots, (a_k, b_k)$. Эти пары он называет красивыми, их множество мы обозначим как P . Теперь представим ряд камней, которые продают купцы, в виде строки S длины n из строчных букв латинского алфавита. Шах считает число таких пар (i, j) , что $1 \leq i < j \leq n$, а камни S_i и S_j образуют красивую пару, то есть существует такое число $1 \leq q \leq k$, что $S_i = a_q$ и $S_j = b_q$.

Если число таких пар оказывается достаточно большим, то шах покупает все камни. Однако в этот раз купцы привезли настолько много камней, что шах не может посчитать это число. Поэтому он вызвал своего визиря и поручил ему этот подсчет. Напишите программу, которая находит ответ на эту задачу.

- **Формат ввода / входного файла (input.txt).** Первая строка входного файла содержит целые числа n и k ($1 \leq n \leq 100000, 1 \leq k \leq 676$) – число камней, которые привезли купцы и число пар, которые шах считает красивыми. Вторая строка входного файла содержит строку S , описывающую типы камней, которые привезли купцы.

Далее следуют k строк, каждая из которых содержит две строчных буквы латинского алфавита и описывает одну из красивых пар камней.

- **Формат вывода / выходного файла (output.txt).** В выходной файл выведите ответ на задачу – количество пар, которое должен найти визирь.
- Ограничение по времени. 1 сек.

- Ограничение по памяти. 64 мб.

- Примеры:

№	input.txt	output.txt
1	7 1 abacaba aa	6
2	7 3 abacaba ab ac bb	7

Листинг кода

```
from gems_additional_classes import *
from lab6.generator import gems_generator
from measurements import *

@measure_perf
def beautiful_pairs(gems, pairs_list):
    ht = BeautifulHashTable(gems)
```

```

        return ht.count_pairs(pairs_list)

if __name__ == "__main__":
    gems_generator(10**5, 676)
    with open('input.txt') as f:
        n, k = map(int, f.readline().split())
        gems_str = f.readline().strip()
        pairs = []
        for _ in range(k):
            p = list(f.readline().strip())
            pairs.append(tuple(p))
    res = beautiful_pairs(gems_str, pairs)
    with open('output.txt', 'w') as f:
        f.write(str(res))
    print(res)

```

Листинг кода дополнительной функции и класса

```

def gems_generator(n=None, k=None):
    alpha_range = list(range(ord('a'), ord('z')))
    if n is None:
        n = random.randint(1, 10**5)
    if k is None:
        k = random.randint(1, 676)

    with open('input.txt', 'w') as f:
        f.write(f'{n} {k}')
        gems_str = "".join(map(chr, random.choices(alpha_range, k=n)))
        f.write(f'\n{gems_str}')
        for _ in range(k):
            pair = "".join(map(chr, random.choices(alpha_range, k=2)))
            f.write(f'\n{pair}')
    return None

```

```

class BeautifulHashTable:
    def __init__(self, s: str):
        self.s = s
        self.pos = [[] for _ in range(26)]
        self._preprocess()

    def _preprocess(self):
        for idx, ch in enumerate(self.s):
            self.pos[ord(ch) - 97].append(idx)

    def count_pairs(self, queries: list[tuple[str, str]]) -> int:
        ans = 0
        for a, b in queries:
            ia = ord(a) - 97
            ib = ord(b) - 97

            A = self.pos[ia]
            B = self.pos[ib]

            i = 0
            lenA = len(A)

```

```

for bj in B:
    while i < lenA and A[i] < bj:
        i += 1
    ans += i
return ans

```

Текстовое объяснение решения

В этой задаче нам нужно посчитать количество «красивых пар» драгоценных камней в строке, где каждая буква обозначает тип камня, а каждая пара (a, b) считается красивой, если первый камень типа a стоит перед вторым типа b. Для этого реализован класс `BeautifulHashTable`, который в конструкторе хранит строку и строит список списков `self.pos` длиной 26, где каждый список содержит индексы всех вхождений соответствующей буквы в строке. Метод `_preprocess()` заполняет эти списки за один проход по строке, обеспечивая сложность $O(n)$. Подсчёт красивых пар реализован в методе `count_pairs`: для каждой пары (a, b) берутся списки индексов A и B и с помощью двух указателей перебираются элементы списка B. Для каждого индекса `bj` из B указатель `i` продвигается по A до первой позиции, которая не меньше `bj`, и значение `i` добавляется к результату, так как оно отражает количество индексов a до текущего индекса b. Такой подход гарантирует, что каждый элемент массивов A и B обрабатывается один раз, что делает алгоритм очень эффективным и укладывающимся в ограничения по времени и памяти. Генератор `gems_generator` используется для тестирования, позволяя быстро создать случайные входные данные. В итоге решение работает корректно и быстро даже для верхней границы входных данных, занимая минимальную память и считая все пары за время, значительно меньшее одной секунды.

Результат работы кода на примерах из задания:

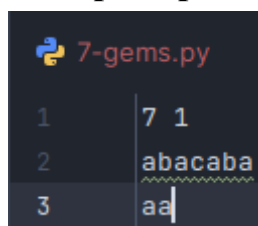


Рисунок 39 – Файл `input.txt` с входными данными из примера 1

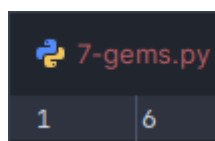
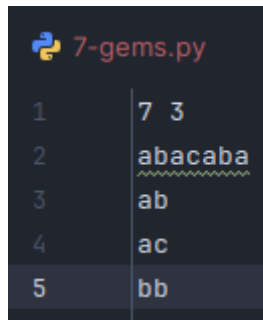
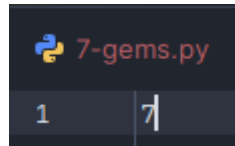


Рисунок 40 – Результат работы кода для данных из примера 1



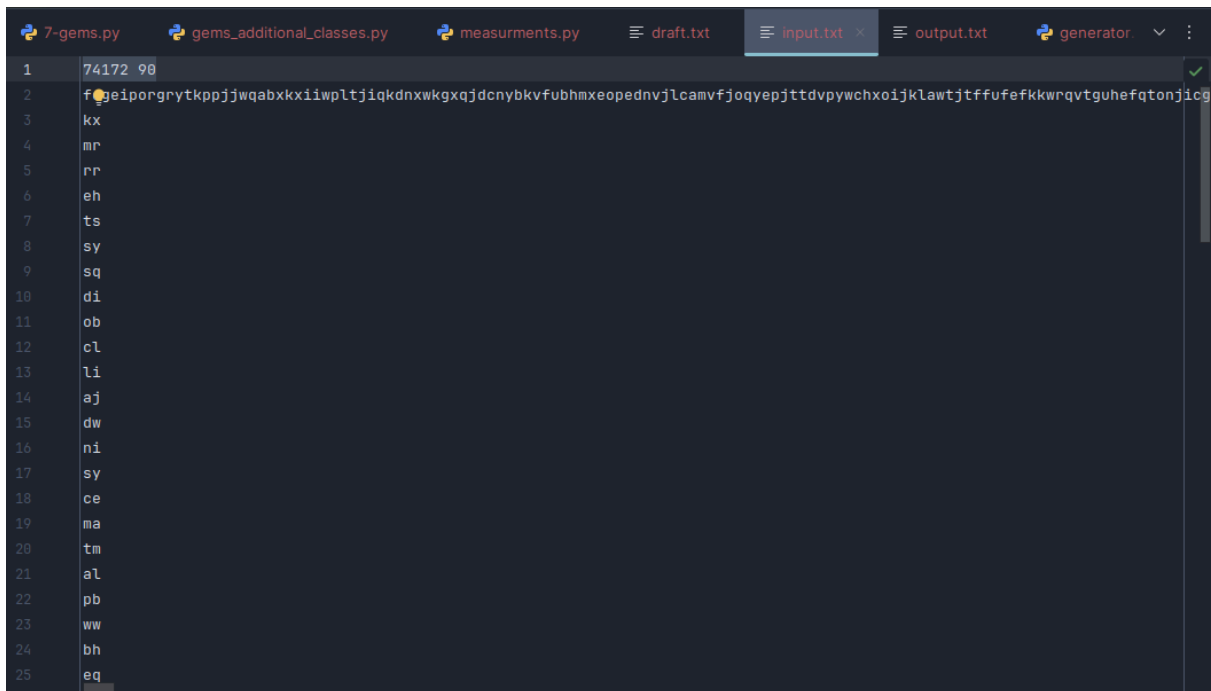
	7-gems.py
1	7 3
2	abacaba
3	ab
4	ac
5	bb

Рисунок 41 – Файл input.txt с входными данными из примера 2



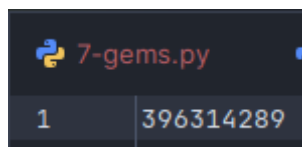
	7-gems.py
1	7

Рисунок 42 – Результат работы кода для данных из примера 2



	7-gems.py
1	74172 90
2	f0eiporgrytkppjjwqabxkxiwplttjiqkdnxwkgxqjdcnybkvfubhmxeopednvlcamvfjoqyepjtttdvpywchxoiijklawtjtffufefkkwrqvtguhefqtonjicg
3	kx
4	mr
5	rr
6	eh
7	ts
8	sy
9	sq
10	di
11	ob
12	cl
13	li
14	aj
15	dw
16	ni
17	sy
18	ce
19	ma
20	tm
21	al
22	pb
23	ww
24	bh
25	eq

Рисунок 43 – Файл input.txt со сгенерированными входными данными (n=74172, k=90)



	7-gems.py
1	396314289

Рисунок 44 – Файл output.txt со сгенерированными входными данными

Результат работы кода на максимальных и минимальных значениях:

7-gems.py	
1	1 1
2	h
3	bm

Рисунок 45 – Минимальное вводимое значение в input.txt

7-gems.py	
1	0

Рисунок 46 – Результат работы кода для значения $n = 1$, записанный в output.txt

```

1 100000 676
2 nxwdqpecsrvgtpdvqlfejslkrainjppgolifjxaplcgyndokrqiuceguphgmuxodoxapmarcindjllpylpimwydaylgrpgkgdkwfupskcdfdosyrtpphghpne
3 oj
4 ni
5 pk
6 js
7 xo
8 qn
9 tj
10 hk
11 mo
12 xq
13 ta
14 is
15 yi
16 li
17 ul
18 go
19 xd
20 fn
21 mm
22 cr
23 cl
24 dp
25 em
  
```

Рисунок 47 – Максимальное вводимое значение в input.txt

7-gems.py	
1	5404169861

Рисунок 48 – Результат работы кода для значения $n = 100000$, $k = 676$, записанный в output.txt

Таблица затрат памяти и времени по заданию 7:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.000005 сек	140.00 MB

Пример 1	0.000006 сек	139.88 MB
Пример 2	0.000007 сек	139.86 MB
Сгенерированный пример	0.021082 сек	141.10 MB
Верхняя граница диапазона значений входных данных из текста задачи	0.193362 сек	142.45 MB

Вывод по задаче: реализованный алгоритм эффективно решает задачу подсчёта красивых пар с высокой производительностью и низким потреблением памяти, полностью удовлетворяя условиям ограничения задачи.

Вывод

В лабораторной работе был рассмотрен алгоритм хэширования и построения хэш-таблиц. С помощью него успешно решаются многие практические задачи с приемлемым временем и потреблением памяти.